

공공데이터포털 오픈 API 테스트

식품안전나라 - 개별기준규격(I2580)

오늘의 학습 목표

- API 상세 페이지에서 요청/응답 구조를 읽을 수 있다.
- 인증키를 `.env` 로 안전하게 관리할 수 있다.
- Python으로 오픈 API를 호출하고 결과를 데이터프레임으로 변환할 수 있다.
- 응답 결과 코드로 성공/실패를 판단할 수 있다.

실습 대상 API

식품안전나라 오픈API - 개별기준규격(I2580)

식품의약품안전처가 제공하는 식품별 **기준규격 정보**(시험항목, 기준값, 단위 등)를 조회하는 API입니다.

- 서비스ID: I2580
- 응답 형식: XML / JSON
- 1회 호출 최대 1,000건

API 상세 페이지에서 먼저 볼 것

API를 호출하기 전, 문서에서 아래 항목부터 확인합니다.

항목	개별기준규격(I2580)
요청 주소	<code>openapi.foodsafetykorea.go.kr/api/...</code>
요청 파라미터	인증키, 서비스ID, 응답형식, 시작/종료위치 (+ 선택 조건)
응답 항목	품목명, 시험항목명, 기준규격, 단위 등
인증 방식	인증키(API Key) 필요
호출 제한	1회 최대 1,000건

1. 요청 URL 구조

```
http://openapi.foodsafetykorea.go.kr/api/{인증키}/{서비스ID}/{요청타입}/{시작위치}/{종료위치}
```

조건을 추가할 때는 뒤에 `/키=값&키=값` 형태로 붙입니다.

```
.../{시작위치}/{종료위치}/PRDLST_CD=값&LAST_UPDT_DTM=값
```

요청 파라미터

순서	이름	필수	설명
1	인증키	O	발급받은 API Key
2	서비스ID	O	I2580 (개별기준규격)
3	요청타입	O	xml 또는 json
4	시작위치	O	조회 시작 행 번호
5	종료위치	O	조회 종료 행 번호
-	PRDLST_CD	X	품목분류코드로 필터링
-	LAST_UPDT_DTM	X	최종수정일(YYYYMMDD)로 필터링

응답 항목 (주요 항목)

필드명	설명
PRDLST_CD	품목분류코드
PRDLST_CD_NM	품목명
TESTITM_NM	시험항목명
SPEC_VAL	기준규격
UNIT_NM	단위명
VALD_BEGN_DT / VALD_END_DT	유효 개시일 / 종료일

전체 응답 항목은 API 상세 페이지의 "출력값(Response Message)" 표에서 확인합니다.

2. 인증키 준비하기

API를 호출하려면 인증키가 필요합니다.

1. 식품안전나라 홈페이지에서 인증키를 신청한다.
2. 이 폴더의 `.env.sample` 을 복사해 `.env` 를 만든다.
3. `FOOD_SAFETY_API_KEY` 값에 발급받은 키를 입력한다.

```
FOOD_SAFETY_API_KEY=발급받은_인증키
```

인증키 발급 절차는 [공공데이터활용 - 인증키 생성.md](#) 참고

3. 코드로 호출하기 - 준비

```
import os, json
import requests
import pandas as pd
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("FOOD_SAFETY_API_KEY", "sample")
```

`.env` 는 깃에 올리지 않고, `.env.sample` 만 공유해서 팀원이 각자 키를 채워 넣게 합니다.

3. 코드로 호출하기 - 요청

```
BASE_URL = "http://openapi.foodsafetykorea.go.kr/api"  
SERVICE_ID, DATA_TYPE = "I2580", "json"  
  
url = f"{BASE_URL}/{API_KEY}/{SERVICE_ID}/{DATA_TYPE}/1/5"  
response = requests.get(url)  
data = response.json()
```

4. 응답 구조 확인하기

응답은 항상 {서비스ID: {...}} 형태입니다.

```
{
  "I2580": {
    "total_count": "12345",
    "row": [ { "PRDLST_CD_NM": "과자", "TESTITM_NM": "산가", "...": "..." } ],
    "RESULT": { "CODE": "INFO-000", "MSG": "정상처리되었습니다." }
  }
}
```

먼저 RESULT.CODE 로 성공 여부를 확인한 뒤 row 를 사용합니다.

5. 데이터프레임으로 변환하기

```
def call_food_api(service_id, api_key=API_KEY, data_type="json",
                  start_idx=1, end_idx=5, **params):
    url = f"{BASE_URL}/{api_key}/{service_id}/{data_type}/{start_idx}/{end_idx}"
    if params:
        url += "/" + "&".join(f"{k}={v}" for k, v in params.items())

    body = requests.get(url).json()[service_id]
    result = body["RESULT"]

    if result["CODE"] != "INFO-000":
        print(f"[{result['CODE']}] {result['MSG']}")
        return result["CODE"], pd.DataFrame()

    return result["CODE"], pd.DataFrame(body["row"])
```

6. 조건 추가해서 검색하기

```
code, df = call_food_api("I2580", end_idx=20)

target_code = df.loc[0, "PRDLST_CD"]
code, df_target = call_food_api("I2580", end_idx=50, PRDLST_CD=target_code)
```

- 응답에서 얻은 `PRDLST_CD` 값을 다음 요청의 조건으로 재사용합니다.
- 파라미터를 `**params` 로 받아두면 다른 API에도 함수를 재사용할 수 있습니다.

7. 결과 코드로 상태 판단하기

코드	의미
INFO-000	정상 처리
INFO-200	해당 조건의 데이터 없음
ERROR-100	인증키가 없거나 유효하지 않음
ERROR-300	필수 파라미터 누락
ERROR-336	요청 범위 초과 (최대 1,000건)
ERROR-500	서버 오류 (요청 형식·서비스ID 오류 포함)

`sample` 키는 데이터셋마다 지원 여부가 다릅니다. 개별기준규격(I2580)은 `sample` 키로는 `ERROR-500` 이 발생하므로 정식 인증키가 필요합니다.

정리

- API 문서는 **요청 URL, 파라미터, 응답 항목, 인증 방식, 호출 제한**을 중심으로 읽는다.
- 인증키는 코드에 직접 적지 않고 `.env` 로 분리해서 관리한다.
- 응답은 `RESULT.CODE` 로 먼저 성공 여부를 확인한 뒤 데이터를 사용한다.
- 하나의 호출 함수를 만들어두면 다른 공공데이터 API에도 재사용할 수 있다.